

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU
CURSO DE CIÊNCIA DA COMPUTAÇÃO

DOCUMENTAÇÃO DE DIÁRIO ALIMENTAR E CONSUMO DE API REST

Desenvolvimento do Módulo de Gerenciamento de Refeições e
Alimentos para o projeto do site Not Fat

Responsável

Eduardo Felipe da Silva Gomes - 01832424

Disciplina: Códigos de Alta Performance - Web
Professor(a): Anderlan Henrique Batista Siqueira

Maceió - AL
2026

Resumo

Este módulo compreende a arquitetura de persistência, controle de estados complexos e consumo de requisições HTTP (*GET*, *POST* e *DELETE*) no módulo de diário nutricional do ecossistema Not Fat. Desenvolvido em React.js, o componente **CardInformativo** opera integrado a serviços REST locais para processar e atualizar, em tempo real, as porções e alimentos associados à rotina de cada usuário.

A principal função desta estrutura é servir como uma interface interativa de controle calórico e macronutricional, utilizando ganchos nativos como **useState** e **useEffect** para gerenciar o ciclo de sincronização de dados síncronos e assíncronos. O tratamento é combinado com o armazenamento em cache local do navegador para indexar as requisições de forma segura através das propriedades de identidade.

Basicamente, a arquitetura de gerenciamento fornecida subdivide-se em:

- **Consumo Assíncrono Adaptativo:** Sincronização dinâmica com tabelas de alimentos e registros individualizados através de *endpoints* escaláveis.
- **Manipulação de Estados Mutáveis:** Controle isolado de buffers locais para filtros de pesquisa, controle de abertura de abas expansíveis e contadores de porções.
- **Persistência e Expurgos Específicos:** Submissão em lote de novos registros nutricionais e deleção direcionada de itens consumidos por refeição.

1 Estrutura e Fluxo de Operação Nutricional

1.1 Gerenciamento de Estados Locais (Hooks)

O mecanismo centraliza múltiplas variáveis reativas que reagem às ações de entrada do usuário na interface e ao tempo de resposta do servidor:

- **alimentosDb**: Vetor que armazena a listagem total de alimentos cadastrados no sistema, servindo de base estática para consultas.
- **itensPorRefeicao**: Estrutura de dicionário chave-valor indexada pelo identificador visual da aba (**index**), otimizando o carregamento sob demanda (*lazy-load*) do diário alimentar.
- **quantidades**: Buffer temporário que rastreia os contadores de porções por alimento antes da confirmação e persistência do lote de dados.

1.2 Ciclo de Vida e Sincronização (Endpoints)

O componente delega o processamento de entrada e saída de dados a funções assíncronas assinaladas por eventos do ciclo de vida ou gatilhos visuais em botões da interface:

1.2.1 Inicialização de Dados (GET /alimentos)

Um gancho de efeito (**useEffect**) isolado com vetor de dependências vazio executa uma requisição assíncrona assim que o componente é montado na tela. A função intercepta a tabela global de alimentos a partir da URI `http://localhost:3000/alimentos` e atualiza o estado **alimentosDb**.

1.2.2 Carregamento Sob Demanda (GET /refeicao/:idUsuario/:idRefeicao)

Diferente da abordagem tradicional, o histórico de consumo não é carregado simultaneamente. O gatilho de execução é acoplado ao clique de expansão da aba (**openIndex**). O sistema lê o cache reativo do **localStorage** para isolar as propriedades através da desestruturação JavaScript:

```
1 const { idUsuario } = JSON.parse(userStorage);
```

Após coletar as *claims*, uma chamada é realizada injetando de maneira dinâmica os parâmetros na URL, cujos dados retornados alimentam de forma exclusiva a chave do índice selecionado no estado de projeção **itensPorRefeicao**.

1.2.3 Persistência de Registros em Lote (POST /adicionar)

A rotina `salvarPorcoesSelecionadas` processa e filtra os buffers que acumularam quantidades superiores a zero. Caso encontre registros válidos, o sistema percorre um laço imperativo (`for...of`) disparando múltiplas requisições assíncronas do tipo `POST`, enviando um corpo no formato *JSON* com as chaves compostas necessárias para vincular a entidade. Ao término da cadeia de promessas, o método recarrega o diário e limpa os buffers de quantidade.

1.2.4 Exclusão Direcionada (DELETE /refeicao/:idUsuario/:idRefeicao/:idAlim

A remoção de um item dispara um método que consome as variáveis de escopo local para requisitar a deleção de um identificador físico no banco de dados. Havendo sucesso (`response.ok`), a aplicação re-executa a sub-rotina de busca para forçar a renderização do estado atualizado.

1.3 Mapeamento Estático de Períodos Alimentares

O sistema adota uma estrutura predefinida e imutável para mapear e segmentar o diário calórico em períodos cronológicos:

Código (<i>ID</i>)	Nome do Período	Tratamento Operacional
1	Desjejum	Primeira refeição basal pós-jejum.
2	Café da manhã	Módulo de aporte matinal padrão.
3	Almoço	Período de grande aporte calórico intermediário.
4	Lanche da tarde	Refeição de fracionamento vespertino.
5	Jantar	Módulo de aporte noturno principal.
6	Ceia	Última janela de macronutrientes do ciclo.

Tabela 1 – Mapeamento Estático de Períodos Nutricionais

1.4 Mecanismos de Controle e Proteção de Erros de Interface

1.4.1 Validação de Escopo Negativo

Para impedir que a manipulação de decremento de porções corrompa o estado para valores matematicamente impossíveis no mundo físico (valores negativos), a função `ajustarQuantidade` envelopa a lógica aritmética pelo método nativo:

```
1 Math.max(0, (prev[nome] || 0) + delta)
```

Isso força um piso absoluto em 0, impedindo a criação de quantidades negativas e estruturando uma barreira de integridade em nível de interface do usuário (*Client-Side Validation*).

1.4.2 Filtragem Reativa Baseada em Texto

A barra de pesquisa localiza os itens na memória do cliente sem necessitar de novas requisições de rede a cada caractere digitado. Ela realiza uma normalização de caixa baixa utilizando o método `.toLowerCase()` combinado com a checagem segura de existência através do operador de encadeamento opcional (`?.`):

```
1 alimento.nome?.toLowerCase().includes(busca.toLowerCase())
```

Essa tratativa impede erros de tempo de execução caso alguma entidade do banco de dados apresente o campo de identificação nominal nulo ou corrompido.